

Atelier — JSON aller-retour

À ouvrir après 1-COURS.pdf du dossier projets/08-json/.

1–3 périodes. Prêt de matériel de l'école.

Le dossier `depart/` de cet exercice contient les fichiers de départ. Vous n'avez pas à ouvrir un terminal.

Où l'on va

Le fichier `data/prets.json` décrit trois matériels. Il garde les données, mais il ne sait pas décider si un appareil est disponible ni changer un prêt. Vous allez transformer chaque ligne en objet `Materiel` : la règle et les actions vivront alors sur l'objet.

Après un clic, la page affichera le nouveau JSON. Le navigateur n'écrit pas directement le fichier sur votre disque : vous copierez ce texte dans `data/prets.json`, puis vous rechargerez la page. Aucun mot de passe ni compte école n'entre dans ce fichier.

À la fin, votre écran ressemble à ceci.

PRÊT ÉCOLE

MatérielLe fichier

SAS · JSON · PRÊT DE MATÉRIEL

On lit le fichier. On rend via l'objet. On réécrit.

Un matériel est disponible si `emprunteur` est une chaîne vide. Le bouton appelle `rendre()` ou `emprunter("Nora")`, puis affiche le JSON à recopier dans `data/prets.json`.

Le matériel

4 objets lus depuis `data/prets.json`.

Vidéoprojecteur salle 2

SALLE
S2
EMPRUNTEUR
—

DISPONIBLE

Emprunter (Nora)

Enceinte Bluetooth

SALLE
Labo
EMPRUNTEUR
Nora

SORTI Rendre

Appareil photo

SALLE
Studio
EMPRUNTEUR
—

DISPONIBLE

Emprunter (Nora)

Pied micro

SALLE
Studio
EMPRUNTEUR

Sur un téléphone de 390 pixels de large, les cartes passent sur une colonne et le JSON reste lisible.



Vous créez exactement ceci

```
sas/
├─ json/
├─ index.html
├─ css/
│  └─ style.css
├─ data/
│  └─ prets.json
├─ js/
└─ materiel.js
```

Le départ contient déjà les quatre fichiers. Le HTML et le CSS sont prêts ; votre travail se fait dans `data/prets.json` et `js/materiel.js`.

Étape 1 — Lire trois descriptions

Ouvrez `sas/json/data/prets.json`. Vous voyez une liste entre crochets et trois groupes entre

accolades. Chaque groupe décrit un matériel avec `nom`, `salle` et `emprunteur`.

Ce JSON est **une liste de données**, pas une classe. Il ne contient ni `estDispo()`, ni `emprunter(qui)`, ni `rendre()` : aucune ligne ne sait agir.

- ☐ Le fichier contient exactement trois descriptions.
- ☐ Le vidéoprojecteur a un emprunteur vide.
- ☐ L'enceinte est empruntée par Nora.
- ☐ Aucun secret n'apparaît dans le fichier.

Ouvrez ensuite `sas/json/index.html` par un double-clic. La page et son style sont déjà en place. Les cartes ne seront justes qu'après avoir complété la classe.

Étape 2 — Fabriquer un seul objet depuis une ligne

La première ligne du JSON décrit un vidéoprojecteur. Une **classe** est le moule ; `new Materiel(...)` produit **un objet réel**, comme un moule à billets produit un billet réel. Cet objet pourra recevoir des questions et des actions que le texte JSON ne connaît pas.

Ouvrez `js/materiel.js`. Le constructeur du départ range déjà les trois valeurs sur l'objet. Complétez seulement `estDispo()` avec la règle nommée **matériel disponible** : elle est vraie quand `emprunteur` est une chaîne vide.

```
return this.emprunteur === ""
```

Rechargez la page, ouvrez la console du navigateur et fabriquez **un seul objet** depuis la première ligne :

```
const video = new Materiel("Vidéoprojecteur salle 2", "S2", "");  
video.estDispo();
```

La console doit répondre `true`. Tapez ensuite `video.nom` : vous parlez à cet objet précis, pas au moule `Materiel`.

Étape 3 — L'état change sur l'objet

Un emprunt change l'emprunteur du matériel concerné. Si le bouton modifie cette propriété lui-même, la règle se disperse dans la page. Les méthodes `emprunter(qui)` et `rendre()` gardent ce changement sur l'objet.

Dans les deux méthodes à trous, écrivez les règles suivantes :

```
emprunter(qui) : this.emprunteur devient qui  
rendre() : this.emprunteur redevient ""
```

Testez le même objet dans la console :

```
video.emprunter("Sam");
video.estDispo();
video.rendre();
video.estDispo();
```

Vous devez obtenir `false`, puis `true`. L'état a changé **sur `video`** ; vous n'avez pas réécrit une ligne du JSON pour fabriquer ce résultat.

Étape 4 — Une ligne devient un objet, puis une carte

Le test portait sur un seul objet. La fonction `charger(lignes)` du départ traite maintenant toute la liste : pour chaque description, elle appelle `new Matériel(ligne.nom, ligne.salle, ligne.emprunteur)`. Le JSON fournit les valeurs ; la classe fournit les règles.

Repérez cette transformation dans `js/materiel.js` :

```
stock = lignes.map(
  (ligne) => new Matériel(ligne.nom, ligne.salle, ligne.emprunteur)
);
```

Puis repérez la boucle `stock.forEach(...)`. Elle demande `item.estDispo()` à chaque objet avant de construire sa carte. Elle ne recalcule jamais `item.emprunteur === ""`.

Rechargez `index.html`. Vous devez voir trois cartes. Le vidéoprojecteur et l'appareil photo sont disponibles ; l'enceinte est sortie.

Étape 5 — Emprunter, puis rendre

Un clic trouve déjà l'objet de sa carte. Si l'objet est disponible, la page lui demande `emprunter("Nora")` ; sinon, elle lui demande `rendre()`. La page orchestre, mais l'objet change son propre état.

Cliquez sur **Emprunter (Nora)** sous le vidéoprojecteur. Sa carte doit afficher « Sorti » et le bloc JSON doit maintenant contenir :

```
"emprunteur": "Nora"
```

Cliquez ensuite sur **Rendre**. La carte redevient disponible et la valeur redevient `""`. Vérifiez dans `js/materiel.js` que le gestionnaire de clic appelle les méthodes sans écrire directement `item.emprunteur = ....`

Étape 6 — Faire l'aller-retour avec le fichier

Le clic a modifié les objets en mémoire. `versJson()` transforme leur état actuel en texte et la page l'affiche dans le bloc blanc. Il reste à reporter ce texte sur le disque.

- 1 Cliquez sur **Emprunter (Nora)** sous le vidéoprojecteur.
- 2 Sélectionnez tout le texte sous « Le JSON à écrire dans le fichier ».
- 3 Remplacez le contenu de `sas/json/data/prets.json` par ce texte.
- 4 Enregistrez, puis rouvrez `data/prets.json` et vérifiez que le vidéoprojecteur contient encore "Nora".
- 5 Si vous utilisez Live Server, rechargez aussi la page : le vidéoprojecteur doit rester sorti.

Si vous ouvrez la page par double-clic, les données initiales viennent du bloc JSON déjà présent dans `index.html` : un rechargement ne relit donc pas le fichier modifié. Avec Live Server, la page relit `data/prets.json`. Dans les deux cas, le fichier ouvert à l'étape 4 est la preuve de l'aller-retour.

Réussi si

- ☐ Vous savez montrer les trois descriptions dans `data/prets.json` et dire pourquoi ce ne sont pas des objets `Materiel`.
- ☐ Vous avez créé un seul `Materiel` en console avant de travailler avec la liste.
- ☐ `estDispo()`, `emprunter(qui)` et `rendre()` sont complétées dans la classe.
- ☐ La boucle demande `estDispo()` aux objets ; elle ne recalcule pas la règle.
- ☐ Un clic passe par `emprunter(qui)` ou `rendre()`.
- ☐ Le nouveau JSON a été copié dans `data/prets.json`, sans secret.
- ☐ La page se lit à 1440 et à 390 pixels de large.

AVEC L'IA

Après vos tests en console, montrez à une IA seulement `charger(lignes)` et la boucle d'affichage. Demandez-lui : « Expliquez en quatre phrases la différence entre une ligne JSON et l'objet `Materiel` construit avec cette ligne. » Comparez sa réponse avec l'endroit où vivent `estDispo()`, `emprunter(qui)` et `rendre()` ; ne lui demandez pas de refaire l'atelier.

POUR LES RAPIDES

Ajoutez une propriété `dateRetour` dans les trois descriptions et dans le constructeur. Écrivez ensuite une méthode `estEnRetard(dateDuJour)` qui compare les dates. La carte doit demander cette nouvelle règle à l'objet, sans la recalculer dans la boucle.

Livrable

Rendez le dossier `sas/json/` avec ses quatre fichiers. Montrez le JSON avant et après un emprunt, puis expliquez sans lire pourquoi la règle de disponibilité vit dans `Materiel` et non dans le fichier JSON.